



High-speed encrypted traffic classification by using payload features

Xingye Yan^{a,b}, Liukun He^c, Yifan Xu^a, Jiuxin Cao^{a,*}, Liangmin Wang^a, Guyang Xie^b

^a School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China

^b Nanjing Huafei Data Technology Co., Ltd., Nanjing 210019, China

^c School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang 212013, China

ARTICLE INFO

Keywords:

Traffic classification
Flow analysis
Information entropy
Machine learning
Randomness test

ABSTRACT

Traffic encryption techniques facilitate cyberattackers to hide their presence and activities. Traffic classification is an important method to prevent network threats. However, due to the tremendous traffic volume and limitations of computing, most existing traffic classification techniques are inapplicable to the high-speed network environment. In this paper, we propose a High-speed Encrypted Traffic Classification (HETC) method containing two stages. First, to efficiently detect whether traffic is encrypted, HETC focuses on randomly sampled short flows and extracts aggregation entropies with chi-square test features to measure the different patterns of the byte composition and distribution between encrypted and unencrypted flows. Second, HETC introduces binary features upon the previous features and performs fine-grained traffic classification by combining these payload features with a Random Forest model. The experimental results show that HETC can achieve a 94% F-measure in detecting encrypted flows and a 85%–93% F-measure in classifying fine-grained flows for a 1-KB flow-length dataset, outperforming the state-of-the-art comparison methods. Meanwhile, HETC does not need to wait for the end of the flow and can extract mass computing features. The average time for HETC to process each flow is only 2 or 16 ms, which is lower than the flow duration in most cases, making it a good candidate for high-speed traffic classification.

1. Introduction

With the rapid development of network communication technologies and protocols, network traffic has become more complex and diverse. Most communication networks use robust encryption algorithms to secure data transmission and prevent privacy leakage [1]. However, encryption also facilitates cyberattackers, who can use it to hide their presence and activities. Encrypted traffic can hide malicious software and other information, resulting in a series of viruses, worms, and Trojans. Consequently, how to classify encrypted traffic becomes an urgent problem that needs to be solved [2].

Traffic Classification (TC) is a foundational technology for identifying sensitive data patterns and categorizing clustered network traffic. TC and its methodologies are of utmost importance in Internet traffic engineering, as they jointly support activities such as network monitoring [3], anomaly detection [4], security assessment, accounting, and service differentiation of utmost importance in Internet traffic engineering, along with related methodologies and tools, as they jointly support activities such as network monitoring [3], anomaly detection [4], se-

curity assessment, accounting, and service differentiation [5]. For these reasons, TC has gained attention from academia and industry in recent years due to the growth in the usage of encryption protocols. The traditional payload-based TC method, namely Deep Packet Inspection (DPI), accurately classifies flows by matching the string information contained in the packet payload with the existing signatures in the database. However, the widespread use of encryption protocols has made DPI techniques ineffective. To compensate for this problem, existing payload-based TC methods focus on detecting the randomness of the payload, as encryption can increase the randomness of the payload. Entropy is a standard metric for detecting the randomness of a sequence [6] and is directly proportional to the randomness. So related studies [7,8] identify whether a flow is encrypted by analyzing the entropy of the payload byte sequence. However, a high entropy does not always mean encryption. Unencrypted compressed flows can also possess high entropies [9], so focusing only on the payload entropy cannot wholly detect encrypted flows from all flows. We need to introduce more representative features. In this scenario, Machine Learning (ML)-based approaches prove to be the most suitable for modern TC, as they

* Corresponding author.

E-mail addresses: 230219449@seu.edu.cn (X. Yan), helk@stmail.uj.s.edu.cn (L. He), xuyifan@seu.edu.cn (Y. Xu), jx.cao@seu.edu.cn (J. Cao), liangmin@seu.edu.cn (L. Wang), mariomaurerm@163.com (G. Xie).

<https://doi.org/10.1016/j.dcan.2024.02.003>

Received 2 July 2022; Received in revised form 21 January 2024; Accepted 20 February 2024

Available online 28 February 2024

2352-8648/© 2024 Chongqing University of Posts and Telecommunications. Production and hosting by Elsevier B.V. on behalf of KeAi Communications Co. Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

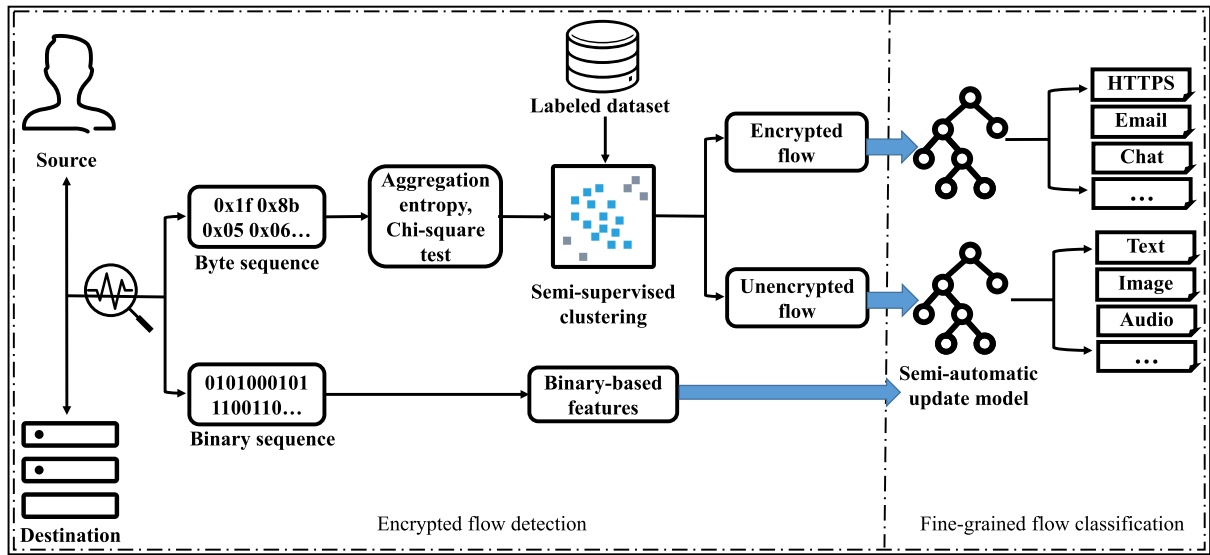


Fig. 1. Architecture of our methodology.

can be used to classify encrypted traffic using side-channel data and efficient ML models. However, their usage relies on acquiring many statistical features. The authors of [2] filtered hundreds of statistical features from a large number of alternative features, and trained a Random Forest (RF) model to classify the encrypted flows. The results show a high classification accuracy. In a high-speed network environment, we can hardly acquire the entire flow due to the continuous high volume of network traffic, which makes most of the statistical features based on the whole flow unavailable. Therefore, it is imperative to propose an effective encrypted TC mechanism suitable for the high-speed network environment by utilizing the low feature requirement of PR-based methods and the effectiveness of ML models.

Two challenges make such TC mechanisms non-trivial. The first challenge is to extract effective features besides entropy features. The range of features we can study is narrow due to calculation limitations and traffic obfuscations [10]. The second challenge is that the presented method should have relatively high classification accuracy for short flows. Short flows contain less information and are more difficult to classify. Having the ability to classify short flows allows a TC method to process more samples per unit of time, which is the most critical ability to realize high-speed encrypted TC.

This research proposes a High-speed Encrypted TC (HETC) method using payload features. HETC focuses on analyzing randomly sampled short flows, and only extracts 4 features to identify the flow types, reducing the classification time. HETC also employs an online flow table to save the analysis results. When an identical flow is captured again, HETC can directly output the flow type to realize high-speed TC. Fig. 1 illustrates the structure of HETC, which contains two stages. In the first stage, we analyze the different patterns of the Byte Composition (BC) and Byte Frequency Distribution (BFD) between encrypted and unencrypted flows. Then we use the aggregation entropy and chi-square test features to characterize this difference. Finally, we combine the features with a semi-supervised clustering-based detection method to effectively detect encrypted flows in all flows, with an F-measure of 94%. In the second stage, we introduce binary features and design a RF semi-automatic update method to distinguish fine-grained flows, such as HTTPS flows in encrypted flows. Compared to other studies, we evaluate HETC on a larger dataset comprising 7 publicly available datasets, covering 12 traffic types. It enables the models to obtain more stable TC results and reduce overfitting. The results show that HETC correctly classifies 85%–93% of fine-grained random flows of size 1 KB, outperforming existing methods.

In summary, the main contributions of this paper are as follows:

- 1) We propose HETC for randomly sampled short flows. HETC can discover the different patterns between encrypted and unencrypted flows at the byte level, and use aggregation entropy with chi-square test features to measure the patterns and distinguish whether the traffic is encrypted.
- 2) By combining the aggregation entropy, chi-square test, binary features, and a RF model, HETC further performs fine-grained TC for refined network management.
- 3) Experiments show that HETC can effectively identify encrypted and unencrypted traffic and achieve fine-grained TC for 1 KB flows, providing support for network resource management and service differentiation in high-speed networks.

The remainder of the paper is organized as follows. Section 2 describes the threat model and related work. Section 3 analyzes the different patterns in BC and BFD between encrypted and unencrypted flows, the features to measure such patterns, and the method to classify the two types of flows. Section 4 introduces the fine-grained flow classification method, and Section 5 describes the experimental preparation, results, and comparative analysis in detail. Finally, Section 6 provides conclusions and future work.

2. Threat model and related work

In this section, we first define the threat model, review existing network TC methods, and divide them into 4 major categories: port-based methods, payload-based methods, ML-based methods, and Deep Learning (DL)-based methods. The advantages and disadvantages of each method are also discussed. As indicated in the introduction, encryption causes a change in the randomness of the packet payload, so we detail one of the payload-based methods, namely the Payload Randomness (PR) method.

2.1. Threat model

In the threat model, we assume that the source and destination are two nodes in the high-speed backbone network, passing encrypted or unencrypted information through multiple routers. The intermediate network managers can locate a bidirectional flow by its IP address, port number, transport layer protocol, and other information. Due to the limitation of storage space and computing resources, it is impossible to store the complete flow and analyze the flow type offline. Network managers can only randomly capture a part of flow fragments to analyze the

flow type, use the online flow table to record the flow information, and output the flow label in real-time when the fragment from the same flow is captured again. The flow table is updated every fixed interval (e.g., 300 s) to ensure that flow records do not expire.

2.2. Traffic classification methods

Port-based methods [11] distinguish traffic by querying a list of well-known ports provided by the Internet Assigned Numbers Authority (IANA), such as port 80 for HTTP and port 443 for SSL. However, with the adoption of dynamic ports and port disguise techniques, the accuracy of this type of method has decreased.

Payload-based methods [12] classify traffic by scanning information contained in the packet payload, extracting signatures, and comparing them with existing signatures in the database. According to the different comparison rules, this paper divides them into two types. The first is DPI, which directly scans every byte of the packet payload and identifies whether the payload contains a specific signature. However, these methods are no longer applicable with the wide employment of encryption protocols such as Transport Layer Security/Secure Socket Layer (TLS/SSL). The other is PR-based methods [13]. These methods do not directly match the specific signature but use the statistical information of the payload to distinguish different types of traffic. PR-based methods can solve the problem that DPI cannot classify encrypted traffic. We will detail this kind of method in the next section.

In recent years, researchers have paid more attention to ML-based methods [14,15], which have high classification accuracy and low resource consumption. Most of the studies extract some representative features from the packets or flows and combine them with ML models such as K-Nearest Neighbor (KNN), Support Vector Machine (SVM), or RF to classify the traffic. Anderson et al. [16] proposed supervised learning models, abbreviated as Mal-I1 and Mal-SVM. The authors used TLS handshake metadata, DNS contexts, and HTTP header information as features to train the l1-logistic regression and SVM models to detect malicious traffic. The results show that Mal-I1 can achieve a 99.978% accuracy at the 0.00% False Discovery Rate (FDR). Dong et al. [17] extracted 15 flow-level features from NetFlow information, including the average packet arrival time, bidirectional packet ratio, and flow duration. They proposed a cost-sensitive SVM called CMSVM to identify the application type of traffic with better classification accuracy than other methods. ML-based approaches extract high-dimensional statistical features from long packet sequences, while PR-based techniques extract fewer features from the payloads of 128 B – 8 KB traffic (about 1 – 10 packets). Our HETC leverages the benefits of PR-based methods. In addition, HETC achieves competitive classification performance by incorporating ML models to detect encrypted traffic with a 94% F-measure.

DL-based methods [18] do not require domain experts to select features, but automatically learn the complex patterns in traffic and extract depth features. Zheng et al. [19] proposed an end-to-end encrypted TC model, BGRUA, which utilized a bidirectional gated recurrent unit to extract features from the raw byte sequences. They then used the attention mechanism to learn weights for the features automatically. BGRUA outperformed multiple state-of-the-art methods. The DeepPacket [20] framework takes raw byte streams as input and trains stacked autoencoders and Convolutional Neural Networks (CNN) to classify network traffic. DeepPacket-CNN has a recall of 94% in the TC tasks. However, most DL-based methods take raw bytes as input, which is time-consuming and violates the limitations of high-speed TC [18]. As a solution, Xie et al. [21] proposed a Self-Attention Method (SAM) for TC. SAM consists of 4 main components: embedding, self-attention, 1D-CNN, and a classifier. SAM uses embeddings to enrich the representation of the packet-level inputs. Then self-attention is leveraged to assign different attention weights to each input part to indicate the importance. 1D-CNN is employed to extract further feature representations. Finally, SAM computes the predicted class probabilities using a

softmax classifier. Regarding the baselines, SAM improves classification accuracy by 13% (traffic type classification) and takes only 2 ms to classify a packet, proving that it can perform online TC at the packet level. We will compare them with our method in the experimental section.

2.3. Payload randomness-based methods

Most existing studies focus on classifying encrypted traffic. However, some communication systems still use plaintext traffic to deliver sensitive information in their actual deployment, and therefore we must first distinguish whether the traffic is encrypted before using an encrypted TC method.

Encryption changes the randomness of the packet payload [22], and most encrypted traffic has a higher degree of randomness than non-encrypted traffic. In information theory, entropy is a measure of data randomness. Related studies first determine whether the traffic is encrypted by calculating the packet payload's entropy, and then analyze the specific type of encrypted traffic.

Mamun et al. [23] first extracted the Bintropy [24], the n-length hexadecimal-encoded entropy, and multiple variants of both entropies of the complete flow. Then they selected the top 8–10 features using a feature selection algorithm and combined the features with a supervised learning model to achieve a classification accuracy of 98.1%.

In high-speed network environments, it is inappropriate for us to calculate the entropy of a complete flow, as sometimes we must detect the flow content on-the-fly. Khakpour et al. [7] proposed Iustitia, a fast content-based flow classifier to leverage the observation that text flows have the lowest entropy, encrypted flows have the highest entropy, and binary flows have intermediate entropies. The basic idea is to classify flows using ML techniques where a feature is the entropy of every specific number of consecutive bytes. Iustitia uses only the first 1 KB of the flow to calculate the entropy and is suitable for high-speed TC environments, but does not consider the presence of compressed flows, which have the same high entropy as encrypted flows.

Casino et al. [9] proposed HEDGE, a high-entropy classifier, to distinguish between encrypted and compressed files. Specifically, if the chi-square test value, the chi-square test percentage confidence, and the number of failed National Institute of Standards and Technology (NIST) randomness tests of a random file fragment pass all pre-defined thresholds, HEDGE considers the file encrypted. The threshold-based strategy makes it possible for HEDGE to classify a file using only 1 feature. However, HEDGE is evaluated on the file dataset, and may be inapplicable to the encrypted TC task, since files are more uniform than the traffic content. Some of the traffic contains both compression and encryption components, such as TLS traffic, which splits the original message into multiple fragments, compresses each fragment separately, and finally encrypts it by the symmetric encryption algorithm. In addition, some encryption protocols have plaintext handshake packets, which may lead to misclassifications.

Doroud et al. [25] proposed an Artificial Neural Network (ANN) classifier based on randomness tests. The ANN extracts 8 randomness test features, such as the Approximate Entropy (AprEnt) test, from the payload of the first 5 packets to classify the flow. Comparison results show that although the randomness test classifier does not perform best, it achieves F1 values greater than 90% at different levels of encrypted traffic detection.

Saleh et al. [26] proposed two systems for classifying encrypted and compressed traffic, namely Engineered Features Classification (EFC) and Raw Data and Engineered Features Classification (RDEFC) systems. EFC extracts Shannon-entropy, chi-square, and Anderson-Darling randomness test features to train a 1D-CNN for classification. RDEFC combines randomness features with load bytes based on EFC and extracts deep representations using 2 1D-CNNs. The results show that EFC achieves a maximum verification accuracy of 81.73%, while RDEFC achieves a higher accuracy of 90.77%.

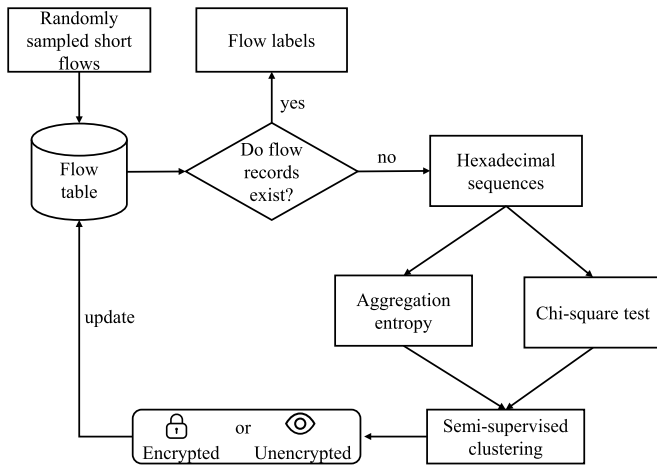


Fig. 2. Encrypted flow detection stage of our methodology.

Based on the previous work, we propose HETC, a high-speed encrypted TC method by using payload features. HETC extracts more useful features by analyzing the differences in the BC, BFD, and binary randomness of different types of flows, achieves higher classification accuracy on short traffic flow datasets, and improves the applicability of PR-based and ML-based encrypted TC methods in high-speed network environments. We detail HETC in the following sections.

3. Feature analysis and encrypted flow detection

This section describes the first phase of HETC, which uses two key features combined with a semi-supervised algorithm to detect encrypted traffic. This strategy allows HETC to utilize many unlabeled samples to assist classification.

Fig. 2 shows an overview of the first stage. After the network probe captures a short flow, HETC computes the flow's quintuple hash to get its flow ID. The corresponding flow label is output if the flow ID appears in the flow table. For flows with no record in the flow table, HETC first strips the TCP or UDP header of each packet to get its transport layer payload. Then HETC converts the payload into a hexadecimal sequence. Subsequently, HETC extracts the aggregation entropy feature from the hexadecimal sequences to classify flows. However, the high-entropy flows contain both encrypted and unencrypted compressed flows. Based on previous work and our evaluation of the BFD of flows, HETC selects the chi-square test feature to distinguish these two high-entropy flows. Finally, HETC combines these two features to propose a semi-supervised clustering-based traffic detection method for encrypted traffic by saving the identification records into the flow table.

3.1. Aggregation entropy feature

According to information theory, the entropy of a random sequence $S = \{e_1, e_2, \dots, e_n\}$ is defined as $-\sum_{i=1}^n p_i \log p_i$, where p_i represents the frequency of the i -th element e_i appearing in the whole sequence. The entropy is a measure of the sequence randomness. The more random the elements in sequence S , the higher the entropy. When there is only 1 element in sequence S , its randomness is minimized and the entropy reaches a minimum value of zero.

In the field of TC, researchers use entropy to distinguish whether the traffic is encrypted. Most existing studies consider every n consecutive bytes in the flow as an element and then calculate the entropy based on the occurrence frequency of each element. However, the method loses part of the encoded information carried by bytes in plaintext flows, as a character may consist of different lengths of bytes given different code tables. For example, a Chinese character occupies two bytes in the Unicode Code Table (CT), while it occupies three bytes in the Unicode

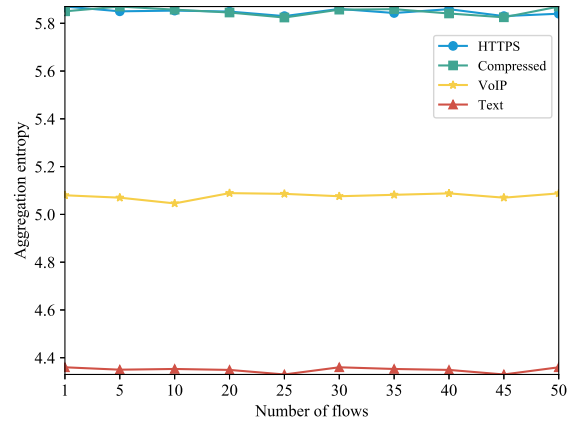


Fig. 3. Comparison of aggregation entropies between encrypted and unencrypted flows.

Transformation Format-8 (UTF8) CT. Therefore, we utilize the encoded information to extract aggregation entropy features based on existing research. The specific process is as follows.

Given a flow sequence $B = \{b_1, b_2, \dots, b_u\}$, we first calculate the CT priority according to the flow destination IP, then initially set the number of consecutive aggregate bytes k to 3. We regard every three consecutive bytes $b_1 b_2 b_3$ as an independent element, look up in the CT by priority, and determine whether $b_1 b_2 b_3$ corresponding to character c exists in the table, such as '\xe3\x80\x82' corresponds to 'o' in UTF8. If it exists, character counter t_c will be increased by 1, and we continue the above procedure by sliding k bytes backward in sequence B . If it does not exist, $k - 1$ consecutive bytes will be regarded an independent element and matched in CTs until $k = 1$.

Then the aggregation entropy of sequence B can be calculated according to

$$H(B) = E(P) = - \sum_{c \in C} (t_c/L) \log_2 (t_c/L) \quad (1)$$

where C represents the characters existing in B , L denotes the total number of characters, and normalization is performed to eliminate the effect of the length of B on the aggregation entropy.

$$D(B) = H(B)/\log_2 m = \sum_{c \in C} (t_c/L) \log_u \frac{1}{(t_c/L)} \quad (2)$$

The details are shown in Algorithm 1. To verify that high entropy and low entropy flows can be effectively classified by the aggregation entropy, we collect 100 encrypted flows (containing 50 HTTPS flows and 50 Voice Over Internet Protocol (VOIP) flows) and 100 unencrypted flows (containing 50 text flows and 50 compressed flows). HTTPS and VoIP flows are derived from the typical encrypted traffic dataset ISCX nonVPN. We generate text files by an automated script and compress text files using the zip algorithm. To better simulate the network traffic environment, we use the sender and receiver hosts on the Internet to replay the encrypted traffic between them. Unencrypted traffic is obtained by fetching these files' upload and download flows. We extract the payload and timestamp of each packet from the traffic and stitch the packet payload according to the timestamp. We will detail the datasets used in subsection 5.2. For each flow, we intercept the first 8 KB bytes to calculate the aggregation entropy and use the tenfold cross-validation method to prevent the influence of outliers. The results are shown in Fig. 3. The HTTPS and compressed flows have the highest aggregation entropies, and the text flows have the lowest entropy, satisfying our expectations.

Algorithm 1 Aggregation entropy feature extraction.

Input: Character counter t_c , byte sequence $B = \{b_1, b_2, \dots, b_u\}$, code table CT , maximum bytes k

Output: Aggregation entropy $D(B)$

```

1:  $CT \leftarrow \text{Prioritization}(CT, \text{desip}), k \leftarrow 3$ 
2: for  $\mu = 1$  to  $u$  do
3:   for  $lct$  in  $CT$  do
4:     if  $lct(B(h, h+k)) \neq \text{None}$  then
5:        $c \leftarrow B(h, h+k)$ 
6:        $t_c \leftarrow t_c + 1$ 
7:       break
8:     end if
9:   end for
10:  if  $B(h, h+k)$  not in  $CT$  then
11:     $k \leftarrow k - 1$ 
12:  end if
13: end for
14:  $D(B) = \sum_{c \in C} (t_c / L) \log_m \frac{1}{(t_c / L)}$ 
15: return  $D(B)$ 

```

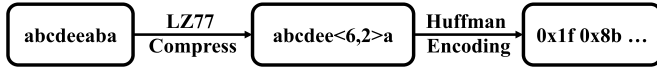


Fig. 4. Principle of the deflate algorithm.

3.2. Chi-square test feature

Using the aggregation entropy, we can divide the flows into two types, high entropy flows and low entropy flows, which means we can detect most unencrypted flows. Still, it is impossible to distinguish between encrypted and unencrypted compressed flows, because compressed data can have high entropies and thus overlap with encrypted data. To further distinguish encrypted flows from compressed flows, we first analyze the deflate algorithm [27], a lossless data compression algorithm that uses both the LZ77 algorithm and Huffman encoding. Deflate is the default algorithm for many standard compressed file formats like zip, gz, 7z, etc.

Fig. 4 represents the compression and encoding processes of the deflate algorithm. In the first stage, deflate uses the LZ77 algorithm to compress plaintext data by eliminating duplicate strings. Specifically, LZ77 first finds the identical string in the previous history for the current string $nstr$. Then LZ77 replaces $nstr$ with $\langle \text{DIS}, \text{LEN} \rangle$, where DIS indicates the distance of $nstr$ from the last repeated string, and LEN denotes the length of $nstr$. For example, string ‘ab’ is transformed into $\langle 6, 2 \rangle$ after compression. To reduce the computation of compressing, LZ77 sets a sliding window with a fixed size of γ KB, which limits the search space for the duplicate string to the first γ KB of data in the current string. In the second stage, all DISs and LENs are processed by Huffman encoding. The sliding window of γ KB theoretically has $\gamma \times 1024$ types of DISs. In reality, the appearance of the repetition phenomenon is localized, and the shorter the DIS, the more frequently it occurs. Therefore, short DISs can be finely encoded so that the code and DIS can be mapped one by one. In contrast, the code for long DISs can be relaxed appropriately, such as in the zip compression format, DISs of 1, 2, 3, 4 are encoded as 0, 1, 2, 3 respectively. In contrast, DISs greater than 769 and less than 1024 are uniformly encoded as 29. The encoding process of LEN is the same. The deflate algorithm converts these codes into binary sequences and stores them in the data to further compress the data size.

We find a certain number of binary sequence repetitions occurs in compressed plaintext data, making some compressed data bytes appear more frequently than others. At the same time, this phenomenon does not appear in encrypted data. We collect 50 TLS-encrypted and 50 zip-compressed flows to verify this observation. For the g -th flow, we intercept the first 8 KB, calculate the frequency distribution $PF_g =$

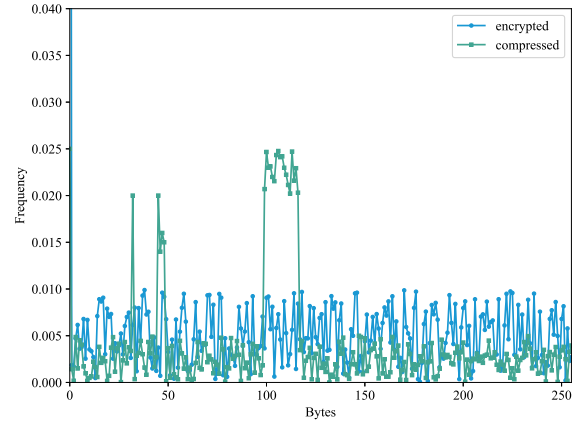


Fig. 5. Comparison of byte frequency distributions between encrypted and compressed flows.

$\{PF_g^1, PF_g^2, \dots, PF_g^{256}\}$ of 256 bytes and finally take the average value.

$$PF = \left\{ \sum_{g=1}^G PF_g^1 / 256, \sum_{g=1}^G PF_g^2 / 256, \dots, \sum_{g=1}^G PF_g^{256} / 256 \right\} \quad (3)$$

Fig. 5 shows a comparison of the BFD of encrypted and compressed flows. The results show that the frequency is uniformly distributed below 0.01 at all bytes in the encrypted flows except zero-byte, where the frequency is higher than 0.02. However, the BFD of the compressed flows is more uneven. For example, the frequency is significantly higher than 0.02 in the range of 100–118 and less than 0.005 from 50–100. To measure the difference between encrypted and compressed flows in BFD, we use the chi-square test to value whether there is a significant difference between the observed and theoretical distributions. Previous work utilized the chi-square test feature to distinguish encrypted data from unencrypted [28] or compressed data [9,29]. For example, Wang et al. [29] used the chi-square test to distinguish encrypted data from encoded (compressed) data to bypass Digital Rights Management (DRM) protection in streaming services. If the observed BFD of the flow is $BF = \{BF_1, BF_2, \dots, BF_\lambda\}$, then its corresponding theoretical BFD is $Q = \left\{ \frac{1}{\lambda}, \frac{1}{\lambda}, \dots, \frac{1}{\lambda} \right\}$. The formula of the chi-square test is shown as follows:

$$\chi^2 = \sum_{\beta=1}^{\lambda} \lambda (BF_\beta - Q_\beta)^2 \quad (4)$$

where λ represents the number of byte types that may appear in the flow. The smaller the chi-square test value is, the closer the BFD of the flow is to a uniform distribution, which means that the flow has a high possibility of belonging to encrypted flows, and vice versa, i.e., belonging to compressed flows.

3.3. Semi-supervised clustering-based method

Since our method runs in a high-speed network environment, we cannot collect entire flows. It is necessary to provide fast responses for short flows. HETC first divides the input traffic into fixed-length short flows according to the 5 tuples, calculates the hash values of the 5 tuples, and determines if there is a corresponding record in the flow table. If the record exists, HETC outputs the flow label directly. If the record does not exist, HETC extracts the aggregation entropy and chi-squared test feature vectors from the labeled and unlabeled flows to create a sample set. Then it divides the samples into different clusters using multiple clustering algorithms. Due to the different types of traffic patterns and the different cluster sizes, densities, and shapes of traffic patterns, no single clustering algorithm can represent all data struc-

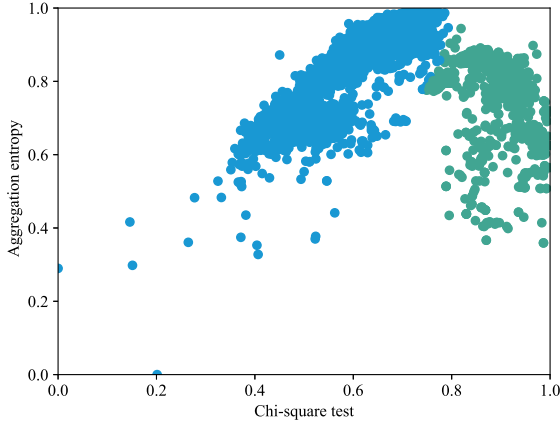


Fig. 6. Clustering result using the K-means algorithm.

tures. To improve the clustering results, HETC uses multiple clustering algorithms that combine the information of a partition set through a consensus function to obtain a more representative partition. Also, the integration process increases the time complexity, so the number of algorithms needs to be considered skillfully. For each cluster containing both labeled and unlabeled samples, HETC uses the label of the largest proportion of samples as its label. Then HETC calculates the probability $p_{f,m} \in [0, 1]$, $f = 1, \dots, F$; $m = 1, \dots, M$ that each type of flow f belongs to a different cluster CR_m , where F and M denote the total numbers of unlabeled samples and clusters, respectively. Subsequently, the weights of different clusters [30] in each clustering result are calculated.

$$w_m = \sum_{v=1}^V H \left(\frac{E_v(CR_1)}{\sum_{j=1}^M E_v(CR_j)}, \dots, \frac{E_v(CR_M)}{\sum_{j=1}^M E_v(CR_j)} \right) \times \left(1 - \left| E_v(CR_m) - \frac{1}{M} \sum_{j=1}^M E_v(CR_j) \right| \right) \quad (5)$$

where CR_m denotes the m -th cluster, $E:CR_m \rightarrow [0, 1]$ represents a mapping relationship, such as the ratio of the cluster size to the total size, V is the optional mapping space, and H denotes the entropy algorithm.

Finally, HETC calculates each clustering result's dynamic weighted probability combination matrix.

$$WP_a = \begin{bmatrix} p_{1,1} \times w_1 & \dots & p_{1,M} \times w_M \\ \vdots & \ddots & \vdots \\ p_{F,1} \times w_1 & \dots & p_{F,M} \times w_M \end{bmatrix} \quad (6)$$

The unlabeled flow sample f is assigned a label according to the maximum strategy.

$$label_f = \arg \max_{mf} \left(\arg \max_{mf_1} WP_1, \dots, \arg \max_{mf_A} WP_A \right) \quad (7)$$

where A represents the number of clustering algorithms used, and mf_a represents the maximum weighted probability label calculated by the a -th clustering algorithm.

Fig. 6 shows the clustering result of the K-means [31] algorithm. The blue and green clusters in the figure represent encrypted and unencrypted flows. The difference between these two types of flows is slight on the vertical axis but more prominent on the horizontal axis, which indicates that the aggregation entropy and chi-square test can effectively detect the encrypted flows from all flows.

4. Fine-grained flow classification

There are multiple types of encrypted or unencrypted flows in the network. Network managers need to provide different handling policies for different kinds of flows. Clustering algorithms usually adopt

similarity measures to divide data points into different subsets [32]. Data within the same subset have high similarity, and data between subsets have low similarity. However, there are subtle differences in the feature distribution of different subclasses under the same broad class, leading to increased similarity. Therefore, to effectively distinguish different subclasses, we design a RF multi-classification model for the fine-grained classification of a given flow. In addition to using the aggregation entropy and chi-square test as input features, we also convert the hexadecimal sequence of the original payload into a binary sequence, and extract two additional binary features from it to improve the model's classification accuracy.

4.1. Binary-based features

4.1.1. Cumulative sum test feature

The NIST SP800-22 specification [33] provides 15 distinct tests to detect randomness in data sequences. These tests are usually used as benchmarks to distinguish between compressed and encrypted data. However, not all tests are suitable for fine-grained flow classification for two reasons. First, most tests cannot provide effective results for high entropy flow classification, and some require extensive calculations. Second, the flow length is relatively short in the high-speed TC environment, and most tests have restrictions on data length. We choose the cumulative sum test as an additional feature for the aggregation entropy and chi-square test, due to its less restriction on data length. The cumulative sum test first replaces 0s with -1s in the binary data, then accumulates from the first to the last bit of the binary to generate q cumulative sum values $\{z_1, z_2, \dots, z_q\}$, and finally calculates $cvalue$, the offset of the maximum cumulative sum z with respect to 0. The closer $cvalue$ is to 0, the greater the randomness of the sequence is. The mathematical definition of $cvalue$ is as follows:

$$cvalue = 1 - \sum_{o=\left(\frac{-z}{z}+1\right)/4}^{\left(\frac{q-1}{z}-1\right)/4} \left[\Phi \left(\frac{(4o+1)z}{\sqrt{q}} \right) - \Phi \left(\frac{(4o-1)z}{\sqrt{q}} \right) \right] + \sum_{o=\left(\frac{-z}{z}+1\right)/4}^{\left(\frac{q-3}{z}-1\right)/4} \left[\Phi \left(\frac{(4o+3)z}{\sqrt{q}} \right) - \Phi \left(\frac{(4o+1)z}{\sqrt{q}} \right) \right] \quad (8)$$

where z denotes the test statistics, i.e., the absolute value of the largest partial sum divided by the square root of the sample size, and Φ is the cumulative distribution function of the standard normal distribution.

4.1.2. Bientropy

Bientropy is a simple entropy algorithm proposed in [34], where the authors consider that the conventional entropy cannot detect periodic binary data. For example, the entropies of binary strings 01011101 and 01010101 are both 1, which completely ignores the periodic nature of the latter. To reflect the periodicity of the binary string, the authors use a weighted average of the Shannon entropies of the string and all but the last binary derivative of the string. When the binary length Θ is lower than 32 bits, Bientropy can be expressed as

$$Bientropy = (1 / (2^{\Theta-1} - 1)) \times \left(\sum_{\theta=0}^{\Theta-2} ((-p(\theta) \log_2 p(\theta) - (1-p(\theta)) \log_2 (1-p(\theta))) 2^\theta) \right) \quad (9)$$

When the binary length n is greater than 32 bits, Bientropy can be reformulated as

$$Bientropy = \left(1 / \sum_{\theta=0}^{\Theta-2} \log_2(\theta+2) \right) \times \left(\sum_{\theta=0}^{\Theta-2} (-p(\theta) \log_2 p(\theta) - (1-p(\theta)) \log_2 (1-p(\theta))) \log_2(\theta+2) \right) \quad (10)$$

4.2. Semi-automatic update method

Considering that the flows in multi-classification tasks may contain unknown types, the false positive rate will increase if the model fails to handle unknown flows. In this subsection, we design a semi-automatic update method to enable the model to identify more types of flows. It includes 3 steps: preliminary classification, uncertainty detection, and structure migration. In the preliminary classification stage, RF, which has performed well in the field of TC, is used as the base model to automatically extract the traffic flow features and provide the preliminary classification probability. If the probability is lower than threshold σ , the model will assign uncertainty labels to the flows and store them locally. These uncertain flows are then marked manually. Finally, the model performs structural migration based on the manually labeled dataset and reclassifies newly collected flows in the network. Each step of the method is described in detail as follows.

4.2.1. Preliminary classification

RF [35] is a classical ML algorithm that integrates multiple decision trees. Suppose there are Ψ samples in the training set and Ω is the feature dimension of a sample. HETC first samples ψ ($\psi < \Psi$) times from these Ψ samples with replacement and uses the ψ samples for tree building. After that, HETC selects ω ($\omega < \Omega$) features from the Ω features and chooses the best splitting points for ω features. The splitting rule is the information gain ratio.

$$Gini(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2 \quad (11)$$

$$Gini_index(D, a) = \sum_b \frac{|D^b|}{|D|} Gini(D^b) \quad (12)$$

where the proportion of samples in the k -th class is p_k . Assume that a discrete attribute a has B possible values $a_1, a_2, \dots, a_B$, and we use a to partition the sample set. In that case, B branch nodes are generated, where the b -th branch node contains all samples in D that take value a_b , denoted as D^b . For a single node in the tree, the splitting will stop when the number of samples in the node is less than a threshold, or the Gini index is greater than a certain threshold, which helps to control the depth of the decision tree and the number of nodes, thus reducing the complexity of the algorithm.

For a newly collected flow sample, each decision tree predicts its type separately, and then RF uses a voting method to determine the final label. RF can run effectively on large datasets and has higher accuracy than other ML algorithms. Therefore, in the preliminary classification stage, we use labeled flow samples to train an RF model to predict unlabeled samples.

4.2.2. Uncertainty detection

The uncertainty detection phase aims to filter out the ambiguous flows using the predicted probability given by the RF model. If the probability is lower than threshold σ , the flow has a high probability of belonging to the unknown types. Therefore, our model first marks it as an uncertain type, stores it locally, and then hands it to human beings for further marking. Although existing studies have proposed many DL automated image labeling methods [36], such methods still adopt supervised learning concepts and cannot be employed to label unknown types of flows. In addition, the authors in [37] proposed an approach based on unsupervised clustering to automatically label packets, which performed well in a small dataset of unknown types of flows. However, due to its simple theoretical background, the clustering algorithm could only effectively mark some flow samples in a massive traffic environment. For instance, K-means cannot determine the number of clusters and is prone to local optima. Therefore, we adopt the manual labeling strategy to ensure the trustworthiness of flow labeling.

4.2.3. Structural migration

After the uncertainty detection phase, the flow dataset $\mathcal{T} = \{(x_\Delta, y_\Delta)\}_{\Delta=1}^\delta$ may contain new types of flows, so the original RF model needs to be updated. Still, there is some similarity in the model structure because the classification problem remain the same. Instead of retraining the model, we update the structure of the model partly using the transfer learning concept. Specifically, we use $\mathcal{T}_{tr} \in \mathcal{T}$ as the training set and the Gini index as the metric to extend each leaf node of the tree down to a complete subtree based on the original structure, so that it can identify the new flow types in the target dataset. Then we use $\mathcal{T}_{te} = \mathcal{T} \setminus \mathcal{T}_{tr}$ as the test set to prune the subtree from the bottom up, i.e., we remove the subtree under each non-leaf node and calculate the current generalization error of the model. If the error does not increase with respect to the error before pruning, we prune the subtree vertically, and if the error grows, we continue to measure another subtree. The generalization error is defined as follows:

$$\frac{1}{|\mathcal{T}_{te}|} \sum_{(x,y) \in \mathcal{T}_{te}} \frac{1}{2} (1 - FUN(pre(x), y))^2 \quad (13)$$

where $|\mathcal{T}_{te}|$ indicates the number of samples in test set $\mathcal{T}_{te}$, and $FUN(pre(x), y)$ denotes the difference between the predicted and actual label. If both labels are the same, the function will return a number greater than 0; otherwise, it returns 0.

$$FUN(pre(x), y) = \begin{cases} 1 & (pre(x) = y) \\ 0 & (pre(x) \neq y) \end{cases} \quad (14)$$

5. Experiment

In this section, we first detail the process of traffic collection and processing, followed by the experimental environment. Finally, we evaluate the classification performance and time consumption of HETC.

5.1. Description of baselines

To present a comprehensive understanding of the effects of HETC, we compare it with the most relevant works, which are summarized as follows:

Three high-speed TC methods, that is Iustitia, HEDGE, and SAM. Iustitia is a framework for identifying high-speed flows, and it is primarily used to distinguish whether a flow is textual, binary, or cryptographic. For Iustitia, we consider its variant using the RF model as a classifier, since the classifier of the initial scheme does not perform as well as RF. HEDGE is an efficient high-entropy traffic classifier to distinguish between compressed and encrypted traffic. SAM is a self-attentive method for TC. It uses packets as inputs and outputs classification results through a neural network, thus fulfilling the requirements of online classification.

Two classical TC methods, namely Mal-SVM proposed in [16] and DeepPacket-CNN proposed in [20]. The former extracts statistical flow features to train the SVM model, and the latter uses raw payload bytes to train the CNN model to classify encrypted traffic.

5.2. Dataset

5.2.1. Traffic collection

Although some encrypted traffic datasets are publicly available in the field of network TC, fewer publicly unencrypted traffic datasets are currently available because the plaintext packet payload may contain personal information about users, such as account passwords, communication messages, etc. To validate the effectiveness of HETC, we must first collect a dataset containing a large amount of labeled traffic to establish a high-speed TC scenario.

We first obtain 5 types of unencrypted files from the Internet, including:

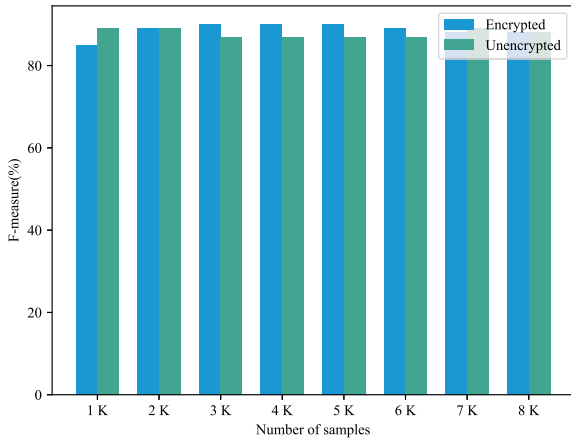


Fig. 7. Performance comparison of fine-grained classification tasks with different numbers of samples.

- 1) Text, 15,000 random texts generated using automated scripts;
- 2) Images, 10,000 images obtained from the ImageNet [38] dataset;
- 3) Audio, 10,000 audio files obtained from the LibriTTS [39] dataset;
- 4) Video, 10,000 video files obtained from the UCF101 [40] dataset;
- 5) Compressed, 15,000 compressed files obtained by compressing text files in the zip format.

Since the actual traffic is obtained in a network environment, we then use the FTP protocol to upload and download these files, and use Wireshark to capture the traffic generated at the network export. As the traffic flows of the core node can provide comprehensive information about the network [41], we also deploy a shunt device at the core of the enterprise network to aggregate HTTP traffic in different fibers, and forward it to local storage through mirroring technology.

For encrypted traffic, we integrate non-DoH normal HTTPS traffic from public dataset DoHBrw-2020 [42] and non-VPN normal encrypted traffic from ISCX VPN2016 [43].

5.2.2. Processing

As for the collected raw traffic, we first remove the retransmission packets caused by network delays, request failure packets caused by server errors, and other damaged or non-artificially generated packets. Then the original traffic is divided into bidirectional flows according to the 5 tuples. In addition, we also strip the handshake packets according to the application layer protocol used. For example, we remove all packets before the second ChangeCipherSpec handshake packet when the flow is encrypted using the TLS protocol. Finally, we trim each flow to a 128 B–8 KB flow fragment to construct 7 fixed-length datasets. To ensure the fairness of the experiment, the numbers of samples of each type of flow are equal, and the ratio of the training set to the test set is 8:2. A detailed description of the dataset is shown in Table 1. There is a data imbalance in different categories of traffic samples, which will lead to the unstable feature distribution in types with small sample sizes, making it challenging to extract effective patterns from them. To solve this problem, we use the strategy of random downsampling, that is, a part of data from the majority set is selected and recombined with the minority set to form a new dataset. In addition, Fig. 7 demonstrates that the classification performance of encrypted flows is high when the number of samples is 2 K and then tends to be stable under the same threshold, while the classification performance of unencrypted traffic is relatively stable. Thus, we choose 2 K and 1 K as the numbers of encrypted and unencrypted flows, respectively.

5.3. Experimental environment

HETC runs on a Linux server with Intel Xeon E5-2630 v3 @ 2.4 GHz CPU and 12 GB RAM. The model is implemented in C++ and relies on

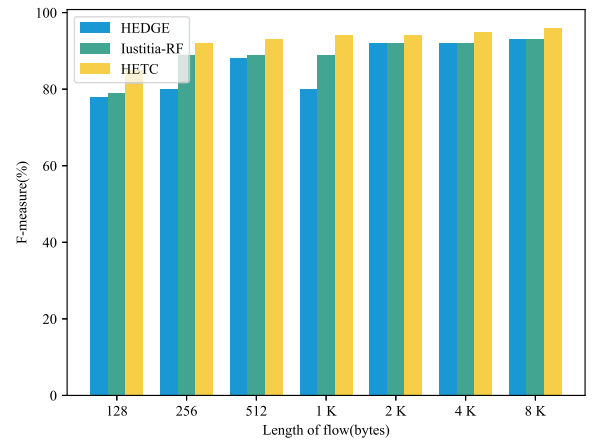


Fig. 8. Performance comparison of different methods for encrypted traffic detection under different flow lengths.

Data Plane Development Kit (DPDK) and other related packet capture libraries, which allows us to achieve 10 Gbps single-port real-time input packet parsing. In addition to that, we use no other external dependency libraries.

5.4. Classification performance on different flows

We evaluate the performance of HETC in different TC tasks and compare it with 5 baseline TC methods, to test how far HETC can shorten the flow length without significantly degrading the performance.

5.4.1. Encrypted flow detection

In the parameter setting of the encrypted flow detection phase, we chose K-means and the Gaussian mixture model as the base clustering algorithms. Fig. 8 shows the F-measure of HETC and other methods for different flow lengths. All 3 methods achieve similar results with an F-measure greater than 94% for long flows (> 2 KB). When the flow length is short (< 1 KB), HETC still maintains a minimum F-measure of 85%, which is significantly better than Iustitia (~5%) and HEDGE (~10%), making it easily applicable to scenarios that require fast classification results such as Quality of Service (QoS) and intrusion detection. The reason is that Iustitia only uses entropy as the criterion for judging whether the flow is encrypted. However, the high-entropy flows contain encrypted and compressed flows. Some image flows also have high entropies. HEDGE has the lowest F-measure among the 3 methods, because HEDGE sets a fixed threshold range as the criteria to measure whether the data is encrypted. When the randomness test value of a flow is not within the set range, the flow is directly determined as encrypted. The advantage of using the threshold-based method is the fast execution. However, the randomness test must meet certain constraints, such as the cumulative sum test requiring the flow length to be greater than 100 bits. When the flow length is short, the randomness test value fluctuates widely above and below the threshold, resulting in a high false positive rate and a decreased F-measure. Therefore, in this paper, we use a combination of the entropy feature and chi-square test to filter out the unencrypted compressed flows further, improving the F-measure and reducing the number of features, with 3 and 2 fewer features compared to HEDGE and Iustitia, respectively.

5.4.2. Fine-grained flow classification

In this subsection, we test the performance of HETC under a finer granularity of flows. Precisely, we classify encrypted flows according to the type of service, such as chat flows, VoIP flows, etc. We classify unencrypted flows according to the content type, such as text flows, compressed flows, etc. Before performing fine-grained classification, we first evaluate the impact of the flow length on the HETC structure. We

Table 1
Properties of the dataset when the flow splitting length is 1 KB.

Type	Source	Original size	Total number of flows(K)	Number of selected samples(K)	Label
HTTPS	DoHBrw-2020	9.4 GB	9626	2	Encrypted
Email	ISCX VPN2016	7.87 MB	8	2	Encrypted
Chat	ISCX VPN2016	28.1 MB	29	2	Encrypted
Streaming	ISCX VPN2016	884 MB	903	2	Encrypted
File Transfer	ISCX VPN2016	12 GB	12984	2	Encrypted
VoIP	ISCX VPN2016	770 MB	798	2	Encrypted
Text	Wireshark capture	220 MB	226	1	Unencrypted
Image	Wireshark capture	102 MB	105	1	Unencrypted
Audio	Wireshark capture	7.4 GB	7620	1	Unencrypted
Video	Wireshark capture	5.1 GB	5232	1	Unencrypted
Compressed	Wireshark capture	83 MB	85	1	Unencrypted
HTTP	Network aggregate	2 GB	2049	1	Unencrypted

Table 2
Comparison of decision tree depths and numbers of nodes under different flow lengths.

Flow length	Tree depth	Tree nodes
128 B	39	410
256 B	30	399
512 B	26	404
1 KB	25	266
2 KB	19	242
4 KB	55	243
8 KB	56	301

find that the shorter the flow length, the deeper the depth of each decision tree in RF, as shown in Table 2. When the flow length is 128 B, the depth reaches a maximum of 39 layers. When the flow length increases to 1 KB, the depth of the decision tree is generally about 25 layers. Compared to the 128 B flow length, the number of leaf nodes at 1 KB is smaller, and the decision tree's complexity is lower. However, this trend is limited. When the flow length exceeds 2 KB, the depth and number of nodes gradually rise again. The RF model consists of multiple decision trees. The decision tree generation process generally starts from the root node, which first selects the corresponding features (e.g., aggregation entropy), followed by selecting splitting points (e.g., aggregation entropy ≤ 5.8 or aggregation entropy > 5.8) for the features. Then it splits the parent node according to the splitting points (e.g., samples with aggregation entropy ≤ 5.8 are categorized in the left child node, and the others are categorized in the right child node). The decision tree will stop splitting if the Gini index exceeds a certain value. As the flows get longer, the average depth of the tree gets smaller, which indicates that the shorter the flow, the more unstable the distribution of extracted features is, resulting in more information provided by each split. Therefore, the tree is deeper. When the flow length exceeds 2 KB, although the depth of the tree is increasing, the overall number of nodes grows slowly, indicating that the distribution of the extracted features is gradually stabilizing.

Considering that the flow length is inversely proportional to the complexity of the HETC model and positively proportional to the time consumption of training and classification, 1 KB is chosen as the flow length in this section, as the complexity and execution time of the model are relatively balanced. Fig. 9 and Table 3 demonstrate the fine-grained flow classification performance when the flow length is 1 KB. Fig. 9 reveals that the overall F-measure of HETC for unencrypted flows is about 5.8% higher than the second-best Iustitia. Although the file structures of the image and compressed flows differ, the F-measure is lower than that

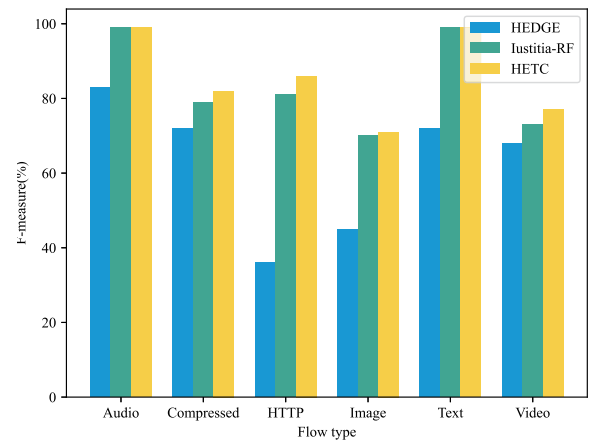


Fig. 9. Unencrypted TC results on an integrated dataset.

on other unencrypted flows. The reason may be that some image files use the same encoding as standard compressed files, e.g., jpeg and zip use Huffman encoding, and png and zip use deflate encoding, resulting in these types of flows having similar feature distributions.

Table 3 demonstrates the performance of different methods for classifying encrypted traffic. The experimental data are obtained by our reproduction experiments, except for DeepPacket, whose results are derived from the original paper. HETC achieves the best TC performance in 3 types of encrypted flows (email, HTTPS, and VoIP), with an F-measure around 95%–99%. The TC performance can be further improved with features extracted from the entire flow, which will increase the execution time. SAM and DeepPacket-CNN achieve the best performance on other flow types. The performance gap is attributed to various factors such as differences in DL platforms and better parameter tuning. Due to the dynamic and varied nature of encrypted traffic, it is necessary to select appropriate models according to the application scenarios.

5.4.3. Unknown type of flow classification

To further verify the classification performance of HETC for unknown types of flows, we select 1 KB as the flow length and construct two new datasets, Denc_1kb and Dunenc_1kb. The training set of Denc_1kb includes all types of encrypted flows except HTTPS and email flow, and the validation and test sets contain all types of encrypted flows. The training set of Dunenc_1kb includes all types of unencrypted flows except text and compressed flows, and the validation and test sets contain all types of unencrypted flows. The ratio of the number of

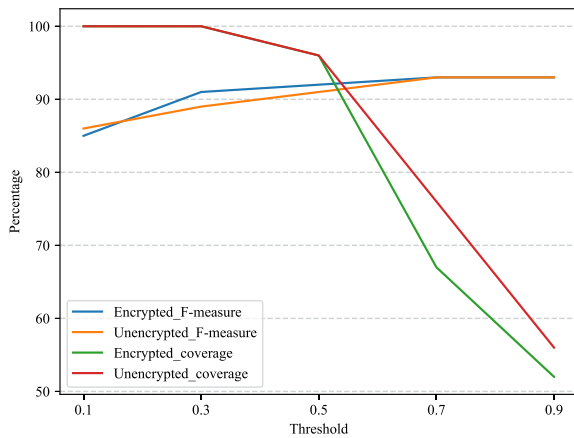
Table 3
Encrypted TC results on ISCX VPN2016.

Type	Chat			Email			File Transfer		
	R. (%)	P. (%)	F1. (%)	R. (%)	P. (%)	F1. (%)	R. (%)	P. (%)	F1. (%)
Mal-SVM	78	64	70	67	78	72	74	74	74
Iustitia-RF (1 KB)	59	68	63	57	64	60	58	88	70
HEDGE (1 KB)	25	42	31	16	42	23	60	72	65
DeepPacket-CNN	71	84	77	87	96	91	100	98	99
SAM	76	68	78	74	88	82	87	96	90
HETC (1 KB)	61	74	71	92	99	95	67	72	69

Type	HTTPS			Streaming			VoIP		
	R. (%)	P. (%)	F1. (%)	R. (%)	P. (%)	F1. (%)	R. (%)	P. (%)	F1. (%)
Mal-SVM	96	99	98	86	72	79	83	77	80
Iustitia-RF (1 KB)	37	48	42	73	42	54	95	86	90
HEDGE (1 KB)	48	36	41	46	45	45	95	75	84
DeepPacket-CNN	-	-	-	87	92	90	88	63	74
SAM	99	99	99	99	99	98	67	91	83
HETC (1 KB)	99	99	99	82	79	80	98	99	99

Table 4
Comparison of classification performance before and after model update.

Evaluation metric	Original		Updated	
	HETC_enc	HETC_unenc	HETC_enc	HETC_unenc
Accuracy (%)	87	86	84	83
Precision (%)	89	87	86	84
F-measure (%)	88	86	85	84
Number of filtered flows	182	181	13	15

**Fig. 10.** Relationship between F-measure, coverage, and threshold.

flows in the training, validation, and test sets is 8:1:1. Fig. 10 illustrates the relationship between the F-measure, coverage, and threshold. As the threshold gradually increases, the performance also rises, and the coverage gradually decreases. The F-measure growth rate is maximized when the threshold is 0.7. Therefore, uncertainty detection threshold σ is set to 0.7. HETC_enc and HETC_unenc are trained using the training sets of Denc_1kb and Dunenc_1kb, respectively. The performance of models is verified using the validation set.

Table 4 shows that the two original models have high classification accuracy overall. Due to uncertainty detection, most of the flow types that do not exist in the training set are filtered out. HETC_enc filters

out 95 HTTPS flows and 87 email flows, and HETC_unenc filters out 85 text flows and 96 compressed flows. These flows are then manually labeled, and RFenc and RFunenc are re-updated using the structure migration strategy provided in the semi-automatic update method. Finally, we re-evaluate HETC utilizing the test set. We can notice that the overall classification accuracy of the two updated models is slightly lower than the original state. Nevertheless, most of unknown types of flows are correctly classified, which verifies that HETC can classify unknown types of flows to some extent.

5.5. Time analysis

To verify that HETC can be applied in high-speed TC environments, we compare the execution time of the 6 TC methods. We experiment on a dataset containing 100,000 encrypted and 100,000 unencrypted flows, and use tenfold cross-validation to obtain stable results. Those flows include unknown types. The purpose of this experiment is mainly to compare the time to process and classify samples by different methods to verify the efficiency of the methods, and therefore HETC outputs its prediction probability regardless of the flow types. Table 5 shows the time required for the methods to classify 1 KB samples. HETC uses different features in the encrypted flow detection and fine-grained classification phases, within a classification time of 2 ms and 16 ms, respectively. In the encrypted flow detection phase, HETC has the same highest speed as SAM, and its detection speed is lower than Iustitia and SAM in the fine-grained classification. Nevertheless, HETC achieves a comparable performance with the state-of-the-art methods, which allows it to run in a high-speed network environment.

Table 5

Comparison of the time required to classify a flow sample in milliseconds.

Method	Mean	Std. dev
Mal-SVM	20	0.007
Iustitia-RF	10	0.006
HEDGE	135	0.008
DeepPacket-CNN	95	0.006
SAM	2	0.006
HETC	2 \ 16	0.004

6. Conclusions and future work

In this paper, we propose HETC, a high-speed encrypted TC method by using payload features. Unlike existing methods, we do not rely on feature selection algorithms to select some effective features from a wide range of alternative features, but consider the high-speed TC environment and comprehensively analyze the different patterns in the BC and BFD between encrypted and unencrypted flows. HETC extracts a few features from randomly sampled short flows to distinguish whether the flow is encrypted and further identifies the exact type it belongs to (e.g., unencrypted text flows, encrypted HTTPS flows). Compared with other methods, HETC has the characteristics of high F-measure (94% for encrypted flow detection, 85%–93% for fine-grained classification) and short execution time (about 2 or 16 ms).

In the future, we plan to investigate new methods in network environments with more traffic types and more heterogeneous traffic distributions, and improve the classification accuracy.

CRediT authorship contribution statement

Xinge Yan: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Supervision, Validation, Writing – original draft, Writing – review & editing. **Liukun He:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Yifan Xu:** Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Writing – original draft, Writing – review & editing. **Jiuxin Cao:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Methodology, Project administration, Resources, Software, Writing – original draft. **Liangmin Wang:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Methodology, Resources, Visualization, Writing – original draft. **Guyang Xie:** Data curation, Formal analysis, Resources, Software, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant No. U1736216.

References

- [1] J. Rathan, K. Vaisla, Vrs-db: preserve confidentiality of users' data using encryption approach, *Digit. Commun. Netw.* 7 (1) (2021) 62–71.
- [2] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, N. Guizani, Optimizing feature selection for efficient encrypted traffic classification: a systematic approach, *IEEE Netw.* 34 (4) (2020) 20–27.
- [3] J. Gallego-Madrid, R. Sanchez-Iborra, P.M. Ruiz, A.F. Skarmeta, Machine learning-based zero-touch network and service management: a survey, *Digit. Commun. Netw.* 8 (2) (2021) 105–123.
- [4] J. Chen, D. Wu, Y. Zhao, N. Sharma, M. Blumenstein, S. Yu, Fooling intrusion detection systems using adversarially autoencoder, *Digit. Commun. Netw.* 7 (3) (2021) 453–460.
- [5] D. Aureli, A. Cianfrani, M. Listanti, M. Polverini, S. Secchi, Augmenting diffserv operations with dynamically learned classes of services, *Comput. Netw.* 202 (2022) 108624.
- [6] G.J. Klir, Uncertainty and information: foundations of generalized information theory, *Kybernetes* 35 (7) (2006) 1297–1299.
- [7] A.R. Khakpour, A.X. Liu, An information-theoretical approach to high-speed flow nature identification, *IEEE/ACM Trans. Netw.* 21 (4) (2012) 1076–1089.
- [8] Y. Wang, Z. Zhang, L. Guo, S. Li, Using entropy to classify traffic more deeply, in: 2011 IEEE Sixth International Conference on Networking, Architecture, and Storage, IEEE, 2011, pp. 45–52.
- [9] F. Casino, K.-K.R. Choo, C. Patsakis, Hedge: efficient traffic classification of encrypted and compressed packets, *IEEE Trans. Inf. Forensics Secur.* 14 (11) (2019) 2916–2926.
- [10] T. Luo, L. Wang, S. Yin, H. Shentu, H. Zhao, Rbp: a website fingerprinting obfuscation method against intelligent fingerprinting attacks, *J. Cloud Comput.* 10 (1) (2021) 1–14.
- [11] G. Aceto, A. Dainotti, W. De Donato, A. Pescapé, Portload: taking the best of two worlds in traffic classification, in: 2010 INFOCOM IEEE Conference on Computer Communications Workshops, IEEE, 2010, pp. 1–5.
- [12] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Muller, K. Hanssger, A survey of payload-based traffic classification approaches, *IEEE Commun. Surv. Tutor.* 16 (2) (2013) 1135–1156.
- [13] J. Zhao, X. Jing, Z. Yan, W. Pedrycz, Network traffic classification for data fusion: a survey, *Inf. Fusion* 72 (2021) 22–47.
- [14] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, J. Aguilar, Towards the deployment of machine learning solutions in network traffic classification: a systematic survey, *IEEE Commun. Surv. Tutor.* 21 (2) (2018) 1988–2014.
- [15] L. Wang, H. Mei, V.S. Sheng, Multilevel identification and classification analysis of tor on mobile and pc platforms, *IEEE Trans. Ind. Inform.* 17 (2) (2020) 1079–1088.
- [16] B. Anderson, D. McGrew, Identifying encrypted malware traffic with contextual flow data, in: Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, ACM, 2016, pp. 35–46.
- [17] S. Dong, Multi class svm algorithm with active learning for network traffic classification, *Expert Syst. Appl.* 176 (2021) 114885.
- [18] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: an overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81.
- [19] X. Liu, J. You, Y. Wu, T. Li, L. Li, Z. Zhang, J. Ge, Attention-based bidirectional gru networks for efficient https traffic classification, *Inf. Sci.* 541 (2020) 297–315.
- [20] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: a novel approach for encrypted traffic classification using deep learning, *Soft Comput.* 24 (3) (2020) 1999–2012.
- [21] G. Xie, Q. Li, Y. Jiang, Self-attentive deep learning method for online traffic classification and its interpretability, *Comput. Netw.* 196 (2021) 108267.
- [22] L. Nian-Sheng, Pseudo-randomness and complexity of binary sequences generated by the chaotic system, *Commun. Nonlinear Sci. Numer. Simul.* 16 (2) (2011) 761–768.
- [23] M.S.I. Mamun, A.A. Ghorbani, N. Stakhanova, An entropy based encrypted traffic classifier, in: International Conference on Information and Communications Security, Springer, 2015, pp. 282–294.
- [24] R. Lyda, J. Hamrock, Using entropy analysis to find encrypted and packed malware, *IEEE Secur. Priv.* 5 (2) (2007) 40–45.
- [25] H. Doroud, A. Alaswad, F. Dressler, Encrypted traffic detection: beyond the port number era, in: 2022 IEEE 47th Conference on Local Computer Networks (LCN), IEEE, 2022, pp. 198–204.
- [26] M.M. Saleh, M. Alslaiman, M.I. Salman, B. Wang, Combining raw data and engineered features for optimizing encrypted and compressed Internet of things traffic classification, *Comput. Secur.* 130 (2023) 103287.
- [27] S. Oswal, A. Singh, K. Kumari, Deflate compression algorithm, *Int. J. Eng. Res. Gen. Sci.* 4 (1) (2016) 430–436.
- [28] P. Choudhury, K.P. Kumar, S. Nandi, G. Athithan, An empirical approach towards characterization of encrypted and unencrypted voip traffic, *Multimed. Tools Appl.* 79 (1–2) (2020) 603–631.
- [29] R. Wang, Y. Shoshitaishvili, C. Kruegel, G. Vigna, Steal this movie: automatically bypassing drm protection in streaming media services, in: USENIX Security Symposium, USENIX Association, 2013, pp. 687–702.
- [30] S. Vega-Pons, J. Correa-Morris, J. Ruiz-Shulcloper, Weighted cluster ensemble using a kernel consensus function, in: Iberoamerican Congress on Pattern Recognition, Springer, 2008, pp. 195–202.
- [31] J. MacQueen, et al., Some methods for classification and analysis of multivariate observations, in: Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, University of California Press, 1967, pp. 281–297.

- [32] D.T. Nguyen, L. Chen, C.K. Chan, Clustering with multiviewpoint-based similarity measure, *IEEE Trans. Knowl. Data Eng.* 24 (6) (2011) 988–1001.
- [33] F. Pareschi, R. Rovatti, G. Setti, On statistical tests for randomness included in the nist sp800-22 test suite and based on the binomial distribution, *IEEE Trans. Inf. Forensics Secur.* 7 (2) (2012) 491–505.
- [34] G.J. Croll, BiEntropy – the measurement and algebras of order and disorder in finite binary strings, in: *Scientific Essays in Honor of H Pierre Noyes on the Occasion of His 90th Birthday*, World Scientific, 2013, pp. 48–64.
- [35] G. Biau, E. Scornet, A random forest guided tour, *Test* 25 (2) (2016) 197–227.
- [36] N. Das, S. Chaba, R. Wu, S. Gandhi, D.H. Chau, X. Chu, Goggles: automatic image labeling with affinity coding, in: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, ACM, 2020, pp. 1717–1732.
- [37] J. Zhang, F. Li, F. Ye, H. Wu, Autonomous unknown-application filtering and labeling for dl-based traffic classifier update, in: *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, IEEE, 2020, pp. 397–405.
- [38] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: a large-scale hierarchical image database, in: *2009 IEEE Conference on Computer Vision and Pattern Recognition*, IEEE, 2009, pp. 248–255.
- [39] H. Zen, V. Dang, R. Clark, Y. Zhang, R.J. Weiss, Y. Jia, Z. Chen, Y. Wu, LibriTTS: a corpus derived from LibriSpeech for text-to-speech, in: *Interspeech 2019, ISCA*, 2019, pp. 1526–1530.
- [40] U.C. for Research in Computer Vision, UCF101: a dataset of 101 human actions classes from videos in the wild, <https://www.crcv.ucf.edu/research/data-sets/ucf101/> (Accessed 15 December 2023).
- [41] L. Feng, Y. Yao, L. Wang, G. Min, Multi-timescale and multi-centrality layered node selection for efficient traffic monitoring in sdns, *Comput. Netw.* 198 (2021) 108381.
- [42] M. MontazeriShatoori, L. Davidson, G. Kaur, A.H. Lashkari, Detection of doh tunnels using time-series classification of encrypted traffic, in: *2020 IEEE Intl. Conf. on Dependable, Autonomic and Secure Computing, Intl. Conf. on Pervasive Intelligence and Computing, Intl. Conf. on Cloud and Big Data Computing, Intl. Conf. on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech)*, IEEE, 2020, pp. 63–70.
- [43] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and vpn traffic using time-related, in: *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*, SCITEPRESS, 2016, pp. 407–414.